

Evolucion hacia Arquitecturas Basadas en Microservicios: Caso Practico TiendaTech

Jose Alejandro Lozano Morales

Curso: 7mo A - Aplicaciones Distribuidas

Universidad Tecnica Estatal de Quevedo

[Enlace al repositorio de la tarea, fecha del último commit 13 de junio de 2026](#)

13 de junio de 2026

Resumen

El presente trabajo analiza de forma comparativa la Arquitectura Orientada a Servicios (SOA) y la arquitectura basada en microservicios, aplicando el estudio al caso de TiendaTech, una plataforma nacional dedicada a la comercializacion de componentes de computacion y a la validacion de compatibilidad de hardware. El objetivo es evaluar que estilo arquitectonico responde mejor a las necesidades de escalabilidad, disponibilidad, seguridad, autonomia de despliegue e integracion del proyecto. Para ello, se examinan los principios de SOA, las características de los microservicios, sus diferencias técnicas, sus implicaciones en rendimiento, datos, seguridad y operacion. Finalmente, se propone una arquitectura orientada a eventos para TiendaTech, apoyada en API Gateway, microservicios especializados, mensajería asincrona, patron Saga, Kubernetes, observabilidad distribuida y principios Zero Trust.

Palabras clave: SOA, microservicios, arquitectura distribuida, Kubernetes, API Gateway, TiendaTech, eventos, seguridad, escalabilidad.

Índice

1. Introduccion	3
2. Contexto del proyecto TiendaTech	3

3. Marco teorico: SOA y microservicios	4
3.1. Arquitectura Orientada a Servicios	4
3.2. Arquitectura basada en microservicios	5
4. Comparacion entre SOA y microservicios aplicada a TiendaTech	5
5. Analisis de SOA en TiendaTech	7
5.1. Ventajas de SOA en TiendaTech	7
5.2. Limitaciones de SOA en TiendaTech	7
6. Analisis de microservicios en TiendaTech	8
6.1. Ventajas de los microservicios en TiendaTech	8
6.2. Desafios de los microservicios en TiendaTech	9
7. Arquitectura visual propuesta	9
7.1. Arquitectura SOA para TiendaTech	10
7.2. Arquitectura basada en microservicios para TiendaTech	11
8. Propuesta de arquitectura orientada a eventos	11
8.1. Patron Saga para ordenes, pagos e inventario	12
9. Gestion de datos inteligentes para compatibilidad de hardware	12
10.Orquestacion y despliegue con Kubernetes	13
11.Seguridad en una arquitectura distribuida	13
12.Observabilidad y monitoreo	14
13.Criterios de decision arquitectonica	15
13.1. Discusión: El dilema entre integración y agilidad	16
14.Conclusiones	16

1. Introduccion

Las aplicaciones empresariales modernas requieren arquitecturas capaces de responder a cambios constantes en la demanda, integracion con sistemas externos, despliegues frecuentes y altos niveles de disponibilidad [1]. En este contexto, TiendaTech se plantea como una plataforma de comercio electronico de alcance nacional especializada en la venta de componentes de computacion. Su principal valor diferencial es un configurador inteligente que permite validar la compatibilidad entre piezas de hardware, como procesadores, tarjetas madre, memorias RAM, fuentes de poder, tarjetas graficas y unidades de almacenamiento.

El crecimiento de una plataforma de estas características genera retos importantes. No todos los módulos tienen la misma carga de trabajo ni el mismo nivel de criticidad [2]. Por ejemplo, el catálogo y el configurador de compatibilidad pueden recibir miles de consultas simultáneas, mientras que los procesos de facturación o soporte pueden tener una demanda menor. Esta diferencia obliga a seleccionar una arquitectura que permita escalar componentes específicos sin aumentar innecesariamente toda la infraestructura.

Tradicionalmente, muchas organizaciones han utilizado la Arquitectura Orientada a Servicios, conocida como SOA, para integrar sistemas empresariales mediante servicios reutilizables y contratos estandarizados. Sin embargo, el enfoque de microservicios propone una descomposición más granular, con servicios pequeños, autónomos y desplegados de manera independiente. Esta diferencia es relevante para TiendaTech, ya que su crecimiento requiere agilidad en el despliegue, aislamiento de fallos, escalabilidad horizontal y una gestión de datos más flexible.

El objetivo de este documento es realizar un análisis comparativo profundo entre SOA y microservicios, no solo desde una perspectiva teórica, sino también aplicada al dominio de TiendaTech. Se evalúan sus ventajas, limitaciones, patrones arquitectónicos y retos técnicos, con el fin de justificar una propuesta de arquitectura basada en microservicios y orientada a eventos.

2. Contexto del proyecto TiendaTech

TiendaTech es una plataforma digital enfocada en la comercialización de componentes de computación. A diferencia de una tienda en línea tradicional, el sistema incorpora un configurador de compatibilidad que permite al usuario seleccionar componentes y validar si pueden funcionar correctamente en una misma configuración. Esta funcionalidad exige una arquitectura robusta, debido a que combina consultas de catálogo, reglas técnicas, disponibilidad de inventario, órdenes de compra, pagos electrónicos y notificaciones [1].

Desde el punto de vista funcional, el sistema puede dividirse en los siguientes dominios

principales:

- **Identidad y autenticacion:** gestiona usuarios, sesiones, roles y mecanismos de acceso seguro.
- **Catalogo y busqueda:** administra productos, categorias, características técnicas, precios e indexación de búsqueda.
- **Inventario:** controla existencias, reservas, actualizaciones de stock y disponibilidad por bodega o proveedor.
- **Compatibilidad de hardware:** valida reglas técnicas entre componentes, como sockets, chipsets, consumo energético, tipo de memoria y dimensiones físicas.
- **Ordenes y pagos:** administra carritos, ordenes, transacciones, estados de pago y confirmaciones.
- **Notificaciones:** envía correos, alertas y mensajes sobre compras, cambios de estado o disponibilidad.

Cada dominio posee necesidades distintas. El módulo de compatibilidad requiere capacidad de procesamiento y consultas frecuentes; el módulo de pagos exige seguridad y confiabilidad; el módulo de catálogo necesita alta disponibilidad; y el módulo de inventario debe conservar coherencia ante cambios simultáneos. Por ello, el estilo arquitectónico elegido debe permitir evolucionar cada componente según sus propias necesidades.

3. Marco teórico: SOA y microservicios

3.1. Arquitectura Orientada a Servicios

SOA es un estilo arquitectónico que organiza las funcionalidades empresariales como servicios reutilizables, accesibles mediante interfaces y contratos bien definidos [1]. Su objetivo principal es facilitar la integración entre sistemas heterogéneos y promover la reutilización de capacidades de negocio. En entornos corporativos, SOA suele apoyarse en un Enterprise Service Bus (ESB), encargado de enrutar mensajes, transformar datos, aplicar reglas de negocio y coordinar la comunicación entre aplicaciones.

Entre sus principales ventajas se encuentran la interoperabilidad, la gobernanza centralizada y la reutilización de servicios. Por ejemplo, una empresa puede exponer un servicio de facturación para que sea utilizado por varias aplicaciones internas. Sin embargo, SOA también puede generar dependencia de una capa central de integración. Cuando demasiadas operaciones pasan por el ESB, este puede convertirse en un cuello de botella técnico y organizacional.

3.2. Arquitectura basada en microservicios

La arquitectura de microservicios propone dividir una aplicación en servicios pequeños, autónomos y orientados a capacidades de negocio específicas [2]. Cada microservicio puede tener su propio ciclo de vida, equipo responsable, base de datos y tecnología de implementación. A diferencia de SOA, los microservicios reducen la dependencia de un bus centralizado y favorecen la comunicación mediante APIs ligeras, eventos o colas de mensajería.

Este estilo se asocia con prácticas modernas como integración continua, despliegue continuo, contenedores, orquestación, monitoreo distribuido y automatización de infraestructura [3] [4] [5]. Su principal ventaja es que permite escalar, modificar y desplegar partes específicas del sistema sin afectar a toda la aplicación. No obstante, también introduce complejidad: comunicación remota, consistencia eventual, observabilidad distribuida, seguridad entre servicios y administración de múltiples bases de datos [6].

4. Comparación entre SOA y microservicios aplicada a TiendaTech

La comparación entre SOA y microservicios debe analizarse considerando las necesidades concretas de TiendaTech. Ambos estilos buscan dividir el sistema en servicios, pero difieren en granularidad, autonomía, gobernanza, comunicación, despliegue y gestión de datos.

Tabla 1: Comparación entre SOA y microservicios aplicada a TiendaTech

Criterio	SOA	Microservicios
Unidad de diseño	Servicios empresariales amplios, reutilizables y con contratos definidos.	Servicios pequeños, autónomos y alineados con capacidades específicas del negocio.
Comunicación	Generalmente mediada por un ESB centralizado, con transformación y enrutamiento de mensajes.	Comunicación mediante REST, gRPC, eventos asíncronos o colas de mensajería [1].
Acoplamiento	Puede existir mayor dependencia debido a contratos compartidos y reglas centralizadas.	Bajo acoplamiento; cada servicio puede evolucionar de forma independiente.

Criterio	SOA	Microservicios
Escalabilidad	Puede requerir escalamiento global o vertical, especialmente si depende de una infraestructura central.	Permite escalamiento horizontal y granular por servicio [2].
Despliegue	Los cambios suelen requerir coordinacion entre sistemas y equipos.	Permite despliegues independientes mediante CI/CD [2].
Gestion de datos	Puede utilizar modelos de datos empresariales compartidos.	Cada microservicio puede administrar su propia base de datos o esquema.
Gobernanza	Centralizada, util para organizaciones con politicas estrictas de integracion.	Descentralizada, con mayor autonomia tecnica por equipo o dominio.
Tolerancia a fallos	El ESB puede convertirse en punto critico si no esta correctamente distribuido.	Los fallos pueden aislarse por servicio aplicando patrones de resiliencia.
Seguridad	Seguridad centralizada en puntos de integracion.	Seguridad distribuida: JWT, OIDC, mTLS, politicas de red y Zero Trust [7].
Aplicacion en TiendaTech	Adecuada para integrar sistemas empresariales tradicionales como facturacion, ERP, bancos o proveedores [1].	Mas adecuada para escalar componentes criticos como catalogo, pagos y configurador de compatibilidad [2].

La Tabla 1 muestra que SOA puede resultar util cuando la prioridad es integrar sistemas existentes y controlar reglas desde un punto central. Sin embargo, TiendaTech no solo necesita integracion, sino tambien escalabilidad diferenciada, despliegues rapidos y autonomia por dominio. Por esa razon, los microservicios ofrecen una respuesta mas apropiada para el crecimiento de la plataforma.

Un ejemplo claro es el configurador de compatibilidad. En una arquitectura SOA, esta funcionalidad podria depender de un servicio empresarial amplio o de un ESB que coordine las solicitudes. En cambio, en una arquitectura de microservicios, el configurador puede operar como un servicio independiente, con su propia base de datos de reglas tecnicas o grafo de conocimiento, escalando segun la demanda sin afectar pagos, usuarios o

facturacion.

5. Analisis de SOA en TiendaTech

Una solucion SOA para TiendaTech podria estructurarse alrededor de servicios empresariales amplios, tales como gestion de clientes, gestion de inventario, procesamiento de pagos, facturacion y logistica. Estos servicios se conectarian mediante un ESB que centralizaria el enrutamiento, la transformacion de mensajes y la aplicacion de reglas comunes.

Esta aproximacion tendria sentido si TiendaTech necesitara integrarse principalmente con sistemas heredados o aplicaciones corporativas ya existentes. Por ejemplo, el ESB podria coordinar la comunicacion con bancos, sistemas contables, proveedores logísticos o plataformas de facturacion electronica. En este escenario, SOA aportaria orden, reutilizacion y control centralizado.

No obstante, al crecer hacia un mercado nacional, TiendaTech enfrentaria limites importantes. Si cada consulta al catalogo, cada validacion de compatibilidad y cada proceso de orden debe pasar por un bus central, el ESB podria transformarse en un cuello de botella. Ademàs, los cambios en reglas de transformacion o contratos compartidos podrian requerir aprobaciones y coordinacion entre varios equipos, reduciendo la velocidad de entrega.

5.1. Ventajas de SOA en TiendaTech

- Facilita la integracion con sistemas externos como bancos, proveedores, empresas logísticas o plataformas contables.
- Permite reutilizar servicios empresariales comunes, como facturacion o gestion de clientes.
- Proporciona gobernanza centralizada para contratos, politicas y reglas de negocio.
- Puede ser adecuada en escenarios donde la estabilidad y la interoperabilidad son mas importantes que la velocidad de despliegue.

5.2. Limitaciones de SOA en TiendaTech

- El ESB puede convertirse en un punto unico de congestión o fallo si no se diseña con alta disponibilidad.
- Los despliegues suelen requerir mayor coordinacion entre equipos y sistemas.
- La escalabilidad puede ser menos granular que en una arquitectura de microservicios [2].

- Los cambios en servicios compartidos pueden afectar a multiples modulos.
- La centralizacion puede reducir la autonomia tecnica de cada dominio del sistema.

6. Analisis de microservicios en TiendaTech

La arquitectura basada en microservicios permite que TiendaTech divida su sistema en unidades pequenas y especializadas [2]. En lugar de construir una sola aplicacion grande o depender de un bus central para todas las operaciones, cada microservicio puede encargarse de una capacidad concreta del negocio.

Una propuesta inicial de descomposicion para TiendaTech podria incluir los siguientes servicios:

- **Microservicio de identidad:** autenticacion, autorizacion, tokens JWT, OIDC y control de sesiones.
- **Microservicio de catalogo:** productos, categorias, fichas tecnicas, imagenes y busqueda.
- **Microservicio de inventario:** existencias, reservas, actualizacion de stock y sincronizacion con proveedores.
- **Microservicio de compatibilidad:** validacion de reglas tecnicas entre componentes de hardware.
- **Microservicio de ordenes:** carritos, creacion de ordenes, estados y seguimiento.
- **Microservicio de pagos:** integracion con pasarelas de pago, confirmaciones y reversiones.
- **Microservicio de notificaciones:** correos, alertas y mensajes transaccionales.

Este enfoque favorece la autonomia. Si el configurador de compatibilidad recibe una alta demanda, se puede aumentar el numero de replicas de ese servicio sin escalar todo el sistema. Del mismo modo, si se necesita actualizar la pasarela de pagos, el cambio puede realizarse sin interrumpir el catalogo o el servicio de busqueda.

6.1. Ventajas de los microservicios en TiendaTech

- Permiten escalar horizontalmente los modulos de mayor demanda, como catalogo y compatibilidad [3] [2].
- Facilitan despliegues independientes mediante pipelines de integracion y entrega continua [2].

- Mejoran el aislamiento de fallos: un problema en notificaciones no debería detener las consultas al catálogo.
- Permiten usar tecnologías diferentes según las necesidades de cada dominio.
- Favorecen la evolución del sistema en equipos pequeños, responsables de servicios concretos.

6.2. Desafíos de los microservicios en TiendaTech

- Aumentan la complejidad operativa al existir múltiples servicios, repositorios, bases de datos y despliegues [3] [2].
- Requieren observabilidad distribuida para rastrear solicitudes entre servicios [6].
- Introducen problemas de consistencia de datos, especialmente en órdenes, pagos e inventario.
- Exigen seguridad servicio a servicio, no solo seguridad perimetral [7].
- Pueden generar latencia adicional debido a la comunicación por red y a las interacciones dinámicas entre servicios [6] [2].

7. Arquitectura visual propuesta

Para comprender la diferencia entre ambos enfoques, se presentan dos diagramas conceptuales aplicados a TiendaTech. El primero representa una arquitectura SOA con un ESB centralizado. El segundo muestra una arquitectura basada en microservicios, con API Gateway, servicios independientes y comunicación mediante eventos.

7.1. Arquitectura SOA para TiendaTech

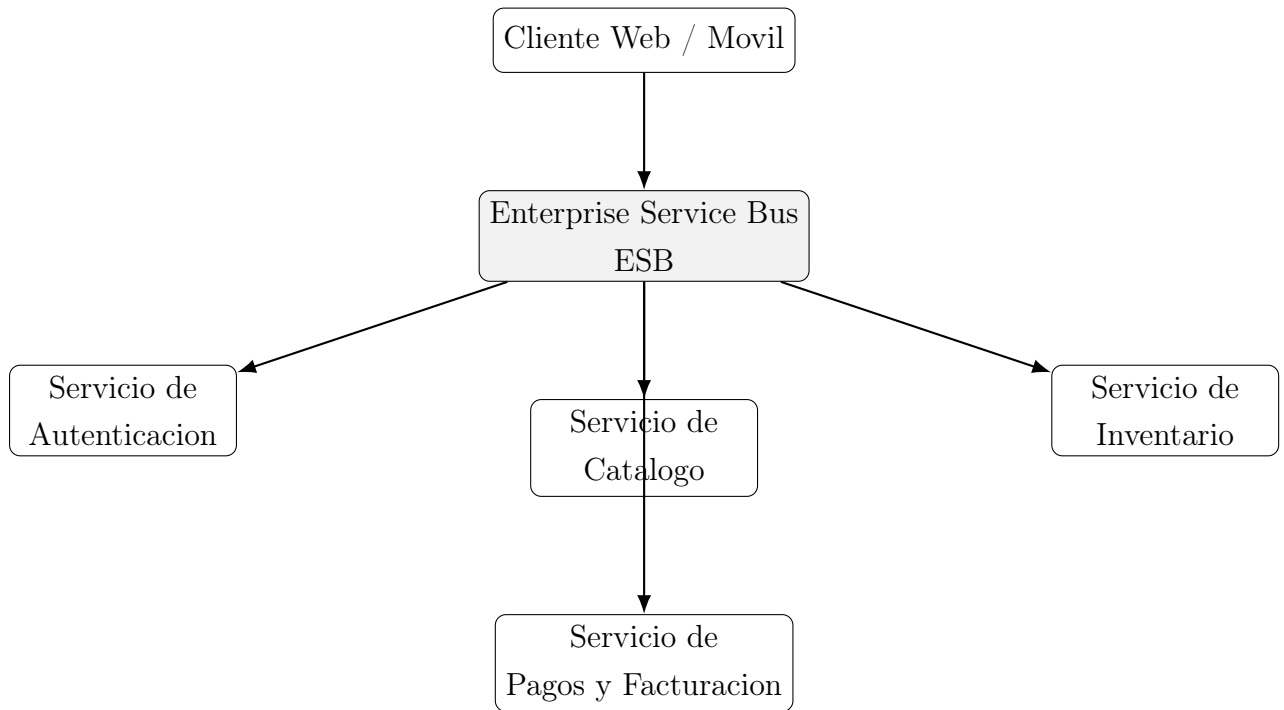


Figura 1: Arquitectura SOA propuesta para TiendaTech

En la Figura 1, el ESB actúa como intermediario principal. Esta estructura facilita integración y control, pero también concentra responsabilidades. Si el bus se satura o falla, varias capacidades del sistema pueden verse afectadas.

7.2. Arquitectura basada en microservicios para TiendaTech

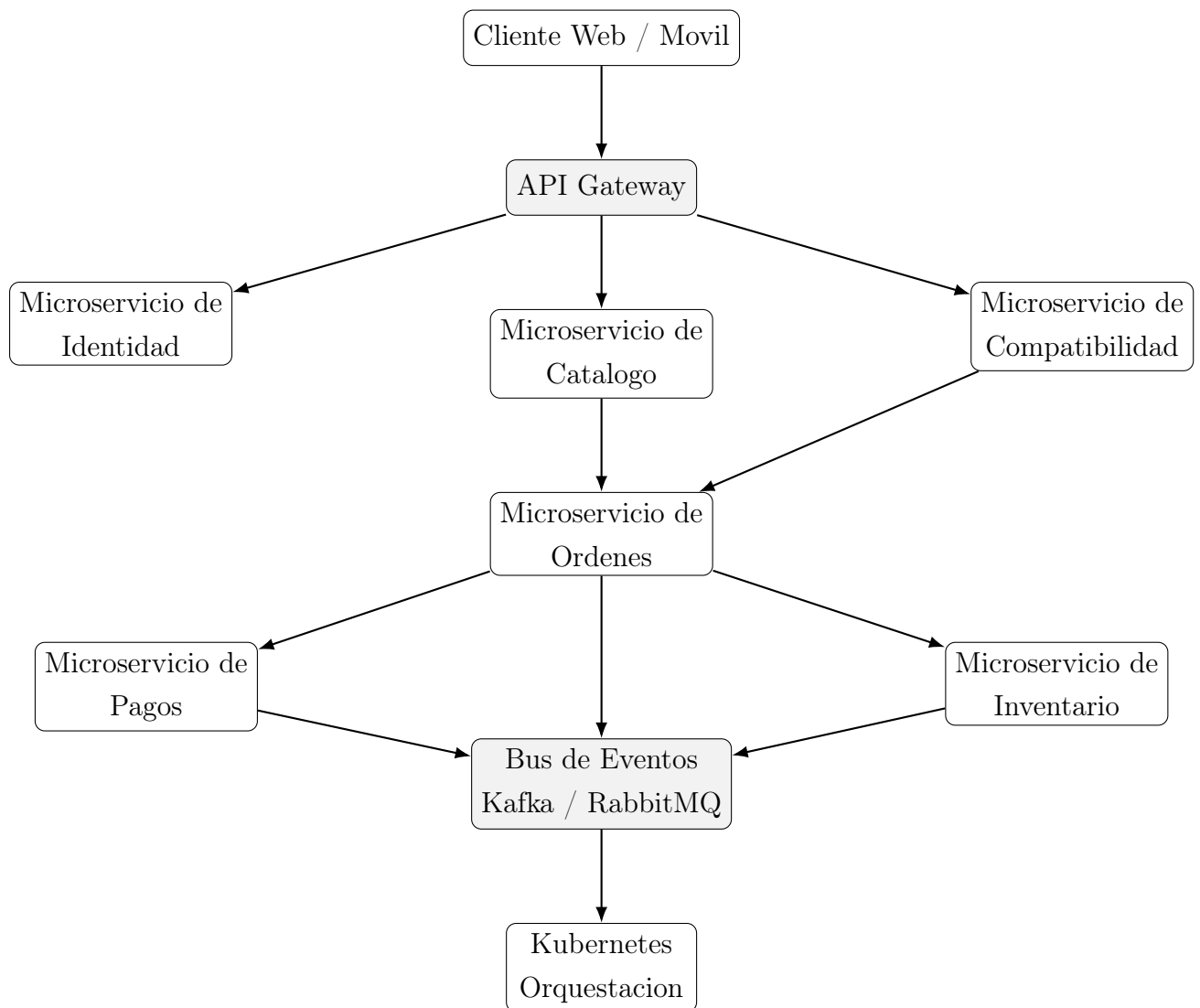


Figura 2: Arquitectura de microservicios propuesta para TiendaTech

En la Figura 2, el API Gateway funciona como punto de entrada para clientes web o móviles, pero no concentra toda la lógica del negocio. Los servicios mantienen responsabilidades separadas y pueden comunicarse mediante APIs o eventos. Esta separación permite que TiendaTech escale el configurador de compatibilidad, el catálogo o los pagos de manera independiente.

8. Propuesta de arquitectura orientada a eventos

Para TiendaTech, una arquitectura de microservicios se fortalece mediante una Arquitectura Orientada a Eventos (EDA) [5]. En este modelo, los servicios no dependen exclusivamente de llamadas sincronicas. En su lugar, publican y consumen eventos como

OrdenCreada, *PagoConfirmado*, *StockReservado* o *CompatibilidadValidada* [2]. Este enfoque reduce el acoplamiento y permite que los servicios reaccionen a cambios sin depender directamente unos de otros [7] [4] [1] [5].

Por ejemplo, cuando el usuario confirma una compra, el servicio de ordenes puede generar un evento *OrdenCreada*. Luego, el servicio de inventario reserva productos, el servicio de pagos procesa la transaccion y el servicio de notificaciones informa al usuario. Si uno de estos servicios falla temporalmente, el sistema puede aplicar reintentos, colas y mecanismos de compensacion.

8.1. Patron Saga para ordenes, pagos e inventario

Uno de los principales desafios de los microservicios es mantener la coherencia de datos sin usar una unica base de datos compartida. En TiendaTech, una compra involucra al menos tres dominios: ordenes, pagos e inventario. Para manejar esta situacion, se propone aplicar el patron Saga, el cual divide una transaccion distribuida en una secuencia de pasos locales con acciones compensatorias.

Un flujo simplificado seria el siguiente:

1. El servicio de ordenes crea una orden en estado pendiente.
2. El servicio de inventario reserva los productos solicitados.
3. El servicio de pagos procesa el cobro.
4. Si el pago se confirma, la orden cambia a estado confirmada.
5. Si el pago falla, se libera el inventario reservado y la orden se marca como cancelada.

Este patron evita depender de transacciones globales bloqueantes y permite que el sistema mantenga consistencia eventual, una propiedad comun en sistemas distribuidos modernos.

9. Gestion de datos inteligentes para compatibilidad de hardware

El configurador de compatibilidad es el componente mas distintivo de TiendaTech [4] [1]. Aunque podria implementarse con una base de datos relacional, el uso de grafos de conocimiento o modelos semanticos permitiria representar relaciones complejas entre componentes. Por ejemplo, una tarjeta madre puede ser compatible con ciertos procesadores segun socket, chipset y version de BIOS; una fuente de poder debe cubrir el consumo estimado de todos los componentes; y un gabinete debe admitir dimensiones especificas de tarjeta grafica y sistema de refrigeracion.

La representacion mediante grafos facilita consultas relacionadas con compatibilidad, dependencia y restricciones [4] [1] [5]. En lugar de evaluar reglas aisladas, el sistema puede recorrer relaciones entre entidades como *Procesador*, *Socket*, *Chipset*, *Memoria*, *Fuente* y *Gabinete*. Esto mejora la precision del configurador y permite ampliar el conocimiento tecnico conforme se agreguen nuevos productos.

En una arquitectura de microservicios, este modelo puede pertenecer exclusivamente al microservicio de compatibilidad. De esta forma, el servicio de catalogo no necesita conocer todas las reglas tecnicas, y el servicio de ordenes solo necesita recibir una respuesta sobre si la configuracion es valida o no.

10. Orquestacion y despliegue con Kubernetes

La adopcion de microservicios requiere una plataforma capaz de gestionar multiples contenedores, replicas, configuraciones y despliegues [3] [4]. Kubernetes es una tecnologia adecuada para este proposito, ya que permite automatizar despliegues, balancear carga, reiniciar contenedores fallidos y escalar servicios segun metricas de uso.

Para TiendaTech, Kubernetes podria aportar las siguientes capacidades:

- **Autoescalado horizontal:** aumentar replicas del catalogo o del configurador cuando crece la demanda [3].
- **Autorreparacion:** reiniciar contenedores que fallen y reemplazar instancias no saludables [3].
- **Despliegues controlados:** aplicar estrategias como rolling updates o canary deployments.
- **Gestion de configuracion:** separar variables de entorno, secretos y configuraciones por ambiente.
- **Aislamiento:** organizar servicios mediante namespaces, politicas de red y limites de recursos.

No obstante, Kubernetes no elimina por si solo la complejidad. Su implementacion debe acompañarse de buenas practicas de observabilidad, seguridad, automatizacion y gestion de costos [3]. De lo contrario, la plataforma puede volverse dificil de operar.

11. Seguridad en una arquitectura distribuida

La migracion hacia microservicios aumenta la superficie de ataque [7]. En una aplicacion monolitica o SOA centralizada, muchas politicas se aplican en puntos de entrada concretos.

En microservicios, en cambio, existen multiples servicios comunicandose por red, lo que exige seguridad en cada interaccion.

Para TiendaTech, la seguridad es critica porque el sistema gestiona datos personales, ordenes de compra, pagos electronicos y posiblemente informacion sensible de clientes. Por ello, se recomienda una estrategia basada en los siguientes principios:

- **Zero Trust:** ningun servicio debe confiar automaticamente en otro por estar dentro de la red interna. Cada solicitud debe autenticarse y autorizarse [7].
- **mTLS entre servicios:** la comunicacion interna debe cifrarse y autenticarse mutuamente.
- **Tokens de corta duracion:** el acceso de usuarios y servicios debe gestionarse mediante mecanismos como JWT y OIDC [7].
- **Gestion segura de secretos:** claves, credenciales y tokens no deben almacenarse en repositorios de codigo. Pueden gestionarse con herramientas especializadas como Vault o mecanismos nativos de la plataforma [1].
- **Políticas de red:** cada servicio debe comunicarse solo con los componentes estrictamente necesarios.

Adicionalmente, un Service Mesh como Istio o Linkerd puede ayudar a aplicar mTLS, politicas de trafico, observabilidad y control de comunicacion entre servicios. Sin embargo, su adopcion debe justificarse segun la madurez del equipo, ya que tambien agrega complejidad operacional.

12. Observabilidad y monitoreo

En arquitecturas distribuidas, los errores no siempre se encuentran en un unico componente [4] [5]. Una solicitud de compra puede pasar por el API Gateway, identidad, catalogo, compatibilidad, ordenes, inventario, pagos y notificaciones. Por ello, TiendaTech necesita observabilidad distribuida [6].

Una estrategia completa debe integrar datos provenientes de multiples servicios y transformarlos en informacion util para monitoreo, analisis predictivo y toma de decisiones operativas [4]:

- **Logs centralizados:** recopilacion de registros de todos los servicios en una plataforma comun.
- **Metrics:** monitoreo de latencia, tasa de errores, consumo de CPU, memoria, solicitudes por segundo y tiempos de respuesta.

- **Trazabilidad distribuida:** seguimiento de una solicitud a través de múltiples servicios.
- **Alertas:** notificaciones automáticas ante errores, caídas, saturación o comportamientos anómalos.

Esta capa es esencial para detectar fallos en pagos, lentitud en el configurador, problemas de inventario o interrupciones en servicios externos. Sin observabilidad, la arquitectura de microservicios se vuelve difícil de diagnosticar y mantener.

13. Criterios de decisión arquitectónica

La elección entre SOA y microservicios no debe basarse en tendencias, sino en criterios técnicos y de negocio. En el caso de TiendaTech, los criterios más importantes son los siguientes:

1. **Escalabilidad diferenciada:** el configurador de compatibilidad y el catálogo requieren escalar de forma independiente [3] [2].
2. **Agilidad de despliegue:** la plataforma necesita actualizar módulos sin detener todo el sistema [2].
3. **Aislamiento de fallos:** un error en notificaciones o búsqueda no debería afectar pagos u órdenes, ya que en sistemas de microservicios los fallos pueden propagarse entre componentes y degradar el rendimiento general del sistema [6] [3] [2].
4. **Complejidad de datos:** la compatibilidad de hardware requiere un modelo flexible y especializado [4] [1] [5].
5. **Seguridad:** los pagos y datos de clientes exigen autenticación, autorización y cifrado en múltiples niveles [7].
6. **Integración externa:** la plataforma debe conectarse con bancos, proveedores, sistemas de envío y posiblemente facturación electrónica, manteniendo interoperabilidad entre sistemas y procesos organizacionales diversos [4] [1] [5].

Con base en estos criterios, SOA puede ser útil para integraciones empresariales externas, pero los microservicios resultan más adecuados para el núcleo transaccional y operativo de TiendaTech. Por ello, una estrategia híbrida también podría considerarse: microservicios para los dominios principales y mecanismos de integración controlados para sistemas externos.

13.1. Discusión: El dilema entre integración y agilidad

La elección arquitectónica implica un balance crítico entre la facilidad de integración de sistemas heredados mediante SOA y la agilidad operativa necesaria para el crecimiento acelerado de TiendaTech. Si bien SOA aporta gobernanza centralizada, esta puede convertirse en un cuello de botella que limita la escalabilidad granular. Por tanto, se discute que la adopción de microservicios traslada la complejidad del código hacia la infraestructura, lo cual exige que la observabilidad distribuida y el modelo Zero Trust sean tratados como requerimientos de diseño fundamentales y no como componentes opcionales.

14. Conclusiones

El análisis comparativo permite concluir que SOA y microservicios comparten la idea de organizar el sistema mediante servicios, pero responden a necesidades distintas. SOA se orienta a la integración empresarial, la reutilización y la gobernanza centralizada. Por su parte, los microservicios priorizan autonomía, despliegue independiente, escalabilidad granular y alineación con dominios específicos del negocio.

En el caso de TiendaTech, la arquitectura basada en microservicios resulta más adecuada para el crecimiento de la plataforma [2]. El configurador de compatibilidad, el catálogo, los pagos y el inventario tienen cargas y requisitos diferentes, por lo que deben poder evolucionar y escalar de manera independiente. Esta necesidad sería más difícil de satisfacer con una arquitectura SOA tradicional basada en un ESB centralizado.

La propuesta recomendada combina microservicios, API Gateway, arquitectura orientada a eventos, patrón Saga, Kubernetes, observabilidad distribuida y seguridad Zero Trust [4] [5]. Esta combinación permite a TiendaTech manejar operaciones críticas, mantener disponibilidad, reducir acoplamiento y mejorar la experiencia del usuario final [1].

No obstante, la migración hacia microservicios no debe asumirse como una solución automática. Requiere madurez técnica, automatización, monitoreo, disciplina en el diseño de APIs, estrategias de consistencia de datos y una gestión rigurosa de seguridad. Si estos elementos se implementan correctamente, TiendaTech puede evolucionar hacia un ecosistema tecnológico robusto, escalable y preparado para competir en un mercado nacional.

Finalmente, desde una perspectiva técnica, la resiliencia y escalabilidad de esta propuesta se sustentan en el uso del patrón Saga para la consistencia de datos, la orquestación mediante Kubernetes y el cumplimiento de un modelo de seguridad Zero Trust. Estas implementaciones, junto con una estrategia híbrida que equilibra la autonomía de microservicios para el núcleo del negocio con la integración externa vía APIs, conforman los pilares fundamentales para garantizar un ecosistema robusto, capaz de evolucionar bajo exigencias de alta disponibilidad.

Declaración de uso de inteligencia artificial

Durante la elaboración de este informe, se empleó inteligencia artificial como soporte en la estructuración del documento, la optimización de la redacción técnica y la generación de la base del código L^AT_EX. Es importante destacar que el proceso de investigación —incluyendo la búsqueda, el análisis crítico de la pertinencia de las fuentes recopiladas y la validación de su aplicabilidad al caso de estudio TiendaTech— fue ejecutado exclusivamente por el autor. Asimismo, todas las decisiones arquitectónicas, resultados y argumentos técnicos presentes en este trabajo han sido verificados rigurosamente, garantizando que no se incluyeran cifras ni afirmaciones que carecieran de respaldo en la literatura académica citada.

Referencias

- [1] Q. An, F. Jiang, A. Neiat, W. Yeoh, K. Venayagamoorthy y A. Zaslavsky, “A Comprehensive Review on IoT Marketplace Matchmaking: Approaches, Opportunities and Challenges,” *ACM Computing Surveys*, vol. 57, n.º 8, págs. 1-35, mar. de 2025. DOI: [10.1145/3715904](https://doi.org/10.1145/3715904).
- [2] M. D. Marieska, A. Yunanta, H. Aulia, A. S. Utami y M. Q. Rizqie, “Performance Comparison of Monolithic and Microservices Architectures in Handling High-Volume Transactions,” *Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi)*, vol. 9, n.º 3, págs. 594-600, jun. de 2025. DOI: [10.29207/resti.v9i3.6183](https://doi.org/10.29207/resti.v9i3.6183).
- [3] Y. Sanjalawe, S. Al-E’mari, S. Fraihat y S. Makhadmeh, “AI-driven job scheduling in cloud computing: a comprehensive review,” *Artificial Intelligence Review*, vol. 58, 2025. DOI: [10.1007/s10462-025-11208-8](https://doi.org/10.1007/s10462-025-11208-8).
- [4] M. Berlato et al., “Digital Platforms for the Built Environment: A Systematic Review Across Sectors and Scales,” *Buildings*, vol. 15, n.º 14, 2025. DOI: [10.3390/buildings15142432](https://doi.org/10.3390/buildings15142432).
- [5] A. Gros, L. De Luca, F. Dubois, P. Véron y K. Jacquot, “From surveys to simulations: Integrating Notre-Dame de Paris’ buttressing system diagnosis with knowledge graphs,” *Automation in Construction*, vol. 170, 2025. DOI: [10.1016/j.autcon.2024.105927](https://doi.org/10.1016/j.autcon.2024.105927).
- [6] S. Zhang et al., “Failure Diagnosis in Microservice Systems: A Comprehensive Survey and Analysis,” *ACM Transactions on Software Engineering and Methodology*, vol. 35, n.º 1, págs. 1-55, dic. de 2025. DOI: [10.1145/3715005](https://doi.org/10.1145/3715005).
- [7] A. Gharib, J. Jaskolka, M. Ibrikahla y A. Matrawy, “Security Management of Horizontal IoT Platforms: A Survey and Comparison,” *ACM Computing Surveys*, vol. 58, n.º 3, págs. 1-39, oct. de 2025. DOI: [10.1145/3766888](https://doi.org/10.1145/3766888).